

SHELL SCRIPT PRACTICAL ASSIGNMENT-1

1. Write a shell script to display all odd and even numbers using various loops available(for, while and until).

echo " Using FOR loop "

```
for i in {1..20}
do
    if [  $((i \% 2)) -eq 0$  ]
    then
        echo "$i is even "
    else
        echo "$i is odd"
    fi
done
```

echo
echo "Using While loop "

```
i=1

while [  $\$i -le 20$  ]
do
    if [  $((i \% 2)) -eq 0$  ]
    then
        echo "$i is even "
    else
        echo "$i is odd"
    fi
    i=$((i+1))
done
echo
```

echo "Using UNTIL loop"

```
i=1
```

```
until [ $i -gt 20 ]
do
  if [ $((i % 2)) -eq 0 ]
  then
    echo "$i is Even"
  else
    echo "$i is Odd"
  fi
  i=$((i+1))
done
```

2. Write a menu driven shell script for basic arithmetic operations.

```
ch=0
while [ $ch -ne 6 ]
do
  echo "1. Addition"
  echo "2. Substraction"
  echo "3. Multiplication"
  echo "4. Division"
  echo "5. Modulus"
  echo "6. Exit"

  echo -n "Enter choice:"
  read ch

  echo -n "Enter no1"
  read no1
  echo -n "Enter no2"
  read no2

  if [ $ch -eq 1 ]; then
    ans=$((no1+no2))
    echo "Ans:$ans"

  elif [ $ch -eq 2 ];then
    ans=$((no1-no2))
    echo "Ans:$ans"

  elif [ $ch -eq 3 ];then
    ans=$((no1*no2))
```

```

        echo "Ans:$ans"

    elif [ $ch -eq 4 ];then
        ans=$((no1/no2))
        echo "Ans:$ans"

    elif [ $ch -eq 5 ];then
        ans=$((no1%no2))
        echo "Ans:$ans"

    elif [ $ch -eq 6 ];then
        echo "Bye bye"
        break

    else
        echo "Invalid choice "
    fi
done

```

3. Write a shell script to generate Fibonacci numbers from 1 to n.

```

echo -n "Enter the number:"
read n

a=0
b=1

for ((i=1; i<=n; i++))
do
    echo -n "$b "
    fn=$((a + b))
    a=$b
    b=$fn
done
echo

```

4. write a shell script to generate prime number from 1 to n, where n any positive integer number by user.

```

echo -n "Enter any positive number:"
read n

```

```
echo "Prime Numbers from 1 to $n are:"
```

```
for ((num=2; num<=n; num++))
do
    flag=0
    for((i=2; i<=num/2; i++))
    do
        if [ $((num % i)) -eq 0 ];then
            flag=1
            break
        fi
    done
    if [ $flag -eq 0 ]
    then
        echo -n "$num "
    fi
done
echo
```

5. Script to display patten.

```
|_
| |_
| | |_
| | | |_
| | | | |_
```

```
for((i=1; i<=5; i++))
do
    for((j=1; j<=i; j++))
    do
        echo -n "|"
    done
    echo "|_"
done
```

6. Write a menu driven shell script for remove, rename, copy and modify a file

```
ch=0
```

```
while [ $ch -ne 5 ]
```

```
do
    echo
    echo "1. Remove a file"
    echo "2. Rename a file"
    echo "3. Copy a file"
    echo "4. Modify(Append) a file"
    echo "5. Exit"
    echo -n "Enter your choice:"
    read ch

    case $ch in
        1)
            echo -n "Enter file name to remove: "
            read file

            if [ -f $file ]
            then
                rm "$file"
                echo "File removed successfully"
            else
                echo "File does not exist."
            fi
            ;;

        2)
            echo -n "Enter old file name: "
            read old
            echo -n "Enter new file name: "
            read new

            if [ -f $old ]
            then
                mv "$old" "$new"
                echo "File renamed successfully"
            else
                echo "File does not exist."
            fi
            ;;

        3)
            echo -n "Enter source file name: "
```

```
read source
echo -n "Enter destination file name: "
read dest
```

```
if [ -f $source ]
then
    cp "$source" "$dest"
    echo "File copied successfully."
else
    echo "File does not exist"
fi
;;
```

4)

```
echo -n "Enter file name to modify: "
read file

if [ -f "$file" ]
then
    echo "Enter text to append:"
    read text
    echo "$text" >> "$file"
    echo "File modified successfully."
else
    echo "File does not exist."
fi
;;
```

5)

```
echo "Exiting....."
;;
```

*)

```
echo "Invalid choice"
;;
esac
```

done

7. Write a script to reverse a six-digit number

```
echo -n "Enter the number "  
read n  
  
rev=0  
while [ $n -gt 0 ]  
do  
    rem=$((n % 10))  
    rev=$((rev * 10 + rem))  
    n=$((n / 10))  
done  
echo "Reverse number is ${rev}"
```

8. Write a menu driven shell script to find area of rectangle, triangle, and circle

```
ch=0  
  
while [ $ch -ne 4 ]  
do  
    echo "-----"  
    echo "1 : Area of Rectangle"  
    echo "2 : Area of Triangle"  
    echo "3 : Area of Circle"  
    echo "4 : Exit"  
    echo "-----"  
    echo -n "Enter your choice: "  
    read ch  
  
    case $ch in  
        1)  
            echo -n "Enter length:"  
            read l  
            echo -n "Enter width:"  
            read w  
            area=$((l * w))  
            echo "Area of Rectangle = $area"  
            ;;  
        2)  
            echo -n "Enter base:"  
            read b
```

```

echo -n "Enter height:"
read h
area=$((b * h / 2))
echo "Area of Triangle = $area"
;;
3)
echo -n "Enter radius:"
read r
area=$(( 22 * r * r / 7))
echo "Area of Circle = $area"
;;
4)
echo "Exiting program..."
;;
*)
echo "invalid choice!"
;;
esac
done

```

9. Write a script to print message like good morning, good afternoon, etc according to the current time.

```

hour=$(date +%H)

if [ $hour -gt 5 ] && [ $hour -lt 12 ]
then
    echo "Good morning"
elif [ $hour -gt 12 ] && [ $hour -lt 17 ]
then
    echo "Good Afternoon"

elif [ $hour -gt 17 ] && [ $hour -lt 21 ]
then
    echo "Good Evening"

else
    echo "Good night "
fi

```

10. Write a menu driven shell script for storing employee information (like add, modify, delete, display info).

```
ch=0
file="emp.txt"

while [ $ch -ne 5 ]
do
    echo "-----"
    echo "1 : Add Employee"
    echo "2 : Modify Employee"
    echo "3 : Delete Employee"
    echo "4 : Display Employees"
    echo "5 : Exit"
    echo "-----"
    echo -n "Enter your choice: "

    read ch
    case $ch in
    1)
        echo -n "Enter Employee id:"
        read id
        echo -n "Enter Employee name:"
        read name
        echo -n "Enter Employee Salary:"
        read salary
        echo "$id | $name | $salary" >> $file
        echo "Employee added Succesfully!"
        ;;
    2)
        echo -n "Enter Employee ID to modify: "
        read id
        if grep -q "$id" $file
        then
            grep -v "$id" $file > text.txt
            mv text.txt $file
            echo "Enter new details"
            echo -n "Name: "
            read name
            echo -n "Salary: "
            read salary
```

```

        echo "$id | $name | $salary" >> $file
        echo "Employee record modified."
    else
        echo "Employee doesn't exist"
    fi
;;
3)
    echo -n "Enter Employee Id to delete:"
    read id

    if grep -q "$id" $file
    then
        grep -v "$id" $file > text.txt
        mv text.txt $file
        echo "Employee deleted successfully."
    else
        echo "Employee not found!"
    fi
;;
4)
if [ -f $file ]
then
    cat $file
else
    echo "File not exist"
fi
;;
5)
echo "Exiting..."
;;
*)
echo "Invalid option"
;;
esac
done

```

11. Write a script to print content of the file if file exists otherwise print appropriate message.

```

echo -n "Enter file name:"
read file

```

```
if [ -f "$file" ];then

echo "File contents"
cat $file

else
    echo "file not found"
fi
```

12. Write a script to check the string entered by user is palindrome or not

```
echo -n "Enter the string:"
read str

rev=""

len=${#str}

for ((i=len-1; i>=0; i--))
do
    rev="$rev${str:i:1}"
done

if [ "$str" = "$rev" ]
then
    echo "It is a Palindrome"
else
    echo "Not a Palindrome"
fi
```

13. Write a script to check given number is prime or not.

```
echo -n "Enter the number:"
read n

flag=0
for ((i=2; i<=n/2; i++))
do
    if [ $((n % i)) -eq 0 ]
    then
```

```

        flag=1
        break
    fi
done

if [ $n -le 1 ]
then
    echo "Not a Prime number"

elif [ $flag -eq 0 ]
then
    echo "Prime Number "
else
    echo "Not a Prime number"
fi

```

14. Write a script to display the student marksheet in appropriate format

```

echo "Enter marks of Subject 1:"
read s1

```

```

echo "Enter marks of Subject 2:"
read s2

```

```

echo "Enter marks of Subject 3:"
read s3

```

```

echo "Enter marks of Subject 4:"
read s4

```

```

echo "Enter marks of Subject 5:"
read s5

```

```

total=$((s1 + s2 + s3 + s4 + s5))
per=$((total / 5))

```

```

echo "-----"
echo "    STUDENT MARKSHEET    "
echo "-----"
echo "Name : $name"
echo "-----"

```

```
echo "Subject 1 : $s1"
echo "Subject 2 : $s2"
echo "Subject 3 : $s3"
echo "Subject 4 : $s4"
echo "Subject 5 : $s5"
echo "-----"
echo "Total Marks : $total / 500 "
echo "Percentage : $per %"
echo "-----"
```

15. Basic salary of a person is input through the keyboard. His dearness allowance is 40% of basic salary and house rent is 20% of basic salary. Write a program to calculate the gross pay.

```
echo -n "Enter the basic salary:"
read salary

da=$((salary * 40 / 100))
hra=$((salary * 20 / 100))

gpay=$((salary + da + hra))

echo "Basic salary: $salary"
echo "DA: $da"
echo "HRA: $hra"
echo "-----"
echo "Gross salary: $gpay"
```

16. The distance between two cities is input through the keyboard. (in km). Write a program to convert this distance into metres, feet, inches and centimetres and display the results.

```
echo -n "Enter distance in kilometers: "
read km

meters=$((km * 1000))
centimeters=$((km * 100000))

feet=$((km * 3281))
inches=$((km * 39370))
```

```

echo "-----"
echo "Distance in Kilometers : $km km"
echo "Distance in Meters : $meters m"
echo "Distance in Centimeters: $centimeters cm"
echo "Distance in Feet : $feet ft"
echo "Distance in Inches : $inches in"
echo "-----"

```

17. The script will receive the filename or filename with its full path, the script should obtain information about this file as given by "ls -l" and display it in proper format.e.g., Filename : , File access permissions : , Number of links : , Owner of the file : , Group to which belongs : Size of file : , File modification date : , File modification time : .

```

echo -n "Enter the file name:"
read file

```

```

if [ -f "$file" ]
then
    info=$(ls -l $file)
    echo $info

    set -- $info

```

```

perm=$1
links=$2
owner=$3
group=$4
size=$5
date=$6
month=$7
time=$8
fname=$9

```

```

echo "-----"
echo "Filename : $fname"
echo "File access permissions : $perm"
echo "Number of links : $links"
echo "Owner of the file : $owner"
echo "Group of the file : $group"
echo "Size of file : $size bytes"

```

```

    echo "File modification date : $date $month"
    echo "File modification time : $time"
    echo "-----"
else
    echo "File doesn't exist"
fi

```

18. If cost price and selling price of an item are entered through the keyboard, write a program to determine whether the seller has made profit or loss. Also determine how much profit/loss is made.

```

echo -n "Enter the buy price:"
read bprice
echo -n "Enter the sell price:"
read sprice

```

```

net=$((sprice - bprice))

```

```

if [ $net -gt 0 ]
then
    echo "Total Profit: $net"

```

```

elif [ $net -lt 0 ]
then
    loss=$((bprice - sprice))
    echo "Total Loss: $loss"
else
    echo "No Profit No Loss"
fi

```

19. The script receives two file names as arguments, the script must check whether the files are same or not, if they are similar then delete the second file.

```

f1=$1
f2=$2

if [ ! -f "$f1" ] || [ ! -f "$f2" ];then
    echo "One or Both files do not exist"
    exit
fi

cmp -s "$f1" "$f2"

```

```

if [ $? -eq 0 ];then
    echo "Files are identical."
    rm "$f2"
else
    echo "Files are different"
fi

```

20. Write a shell script to display the menu driven interface :

1) list all files of the current directory 2) print the current directory 3) print the date 4)print the users otherwise display Invalid Option.

```

echo "1) List all files in current directory"
echo "2) Print current directory"
echo "3) Print date"
echo "4) Print users"
echo "-----"
echo -n "Enter your choice: "
read ch

```

```

case $ch in

```

```

1)

```

```

    echo "Files in Current directory"
    ls
    ;;

```

```

2)

```

```

    echo "Current directory"
    pwd
    ;;

```

```

3)

```

```

    echo "Current Date:"
    date
    ;;

```

```

4)

```

```

    echo "users"
    who
    ;;

```

```
*)
    echo "invalid option"
;;
esac
```

21. Two numbers are entered through the keyboard, find the power, one number raised to another.

```
echo -n "Enter number 1:"
read n1
echo -n "Enter number 2:"
read n2

power=$((n1 ** n2))

echo "Power of $n1 is $power"
```

22. Write a script which has the functionality similar to head and tail commands.

```
echo -n "Enter File name:"
read file

if [ ! -f "$file" ]
then
    echo "File doesn't exist"
    exit
fi

echo "Display first n lines (head)"
echo "Display last n lines (tail)"
echo -n "Enter your choice"
read ch

echo -n "Enter the lines"
read n

case $ch in
1)
    count=0
    while read line
    do
```

```

        echo "$line"
        count=$((count+1))
        if [ $count -eq $n ]
        then
            break
        fi
    done < "$file"
;;
2)
total=$(wc -l < "$file")
start=$((total - n + 1))

while read line
do
    count=$((count+1))
    if [ $count -ge $start ]
    then
        echo "$line"
    fi
done < "$file"
;;
*)
echo "Invalid option"
;;
esac

```

23. The script displays a list of all files in the current directory to which you have read, write and execute permissions.

```

echo "Files with read, write and execute permissions:"

for file in *
do
    if [ -f "$file" ] && [ -r "$file" ] && [ -w "$file" ] && [ -x "$file" ]
    then
        echo "$file"
    fi
done

```

24. The script receives any number of filenames as arguments. It should check whether every argument supplied is a file or directory. If it is a directory, it should be reported. If it is a filename then name of the file as well as the number of lines present in it should be reported.

```
for item in "$@"
do
    if [ -d "$item" ]
    then
        echo "$item is a directory."
    elif [ -f "$item" ]
    then
        lines=$(wc -l < "$item")
        echo "File:$item | Lines: $lines"
    else
        echo "$item is not a file "
    fi
done
```

25. Display the dates falling on Sundays of the current month.

```
echo "Dates falling on Sundays in the current month:"
```

```
cal | awk 'NR>2 {print $1}' | grep -v "^$"
```


SHELL SCRIPT PRACTICAL ASSIGNMENT-2

1. Write a script that takes file name from user and display all line starting with a or b or c. (Use grep/sed)

using grep

echo "Enter file name:"

read filename

if [-f "\$filename"]; then

echo "Lines starting with a, b, or c:"

grep '^[abc]' "\$filename"

else

echo "File not found!"

fi

using sed

echo "Enter file name:"

read filename

if [-f "\$filename"]; then

echo "Lines starting with a, b, or c:"

sed -n '/^[abc]/p' "\$filename"

else

echo "File not found!"

fi

2. Write a script that takes file name from user and display all line starting not with a or b or c. (Use grep/sed)

using grep

echo "Enter file name:"

read filename

```
if [ -f "$filename" ]; then
    echo "Lines NOT starting with a, b, or c:"
    grep '^[^abc]' "$filename"
else
    echo "File not found!"
fi
```

```
# using sed
echo
```

```
echo "Enter file name:"
read filename
```

```
if [ -f "$filename" ]; then
    echo "Lines NOT starting with a, b, or c:"
    sed -n '/^[^abc]/p' "$filename"
else
    echo "File not found!"
fi
```

3. Write a script that takes file name from user and display all line starting not with a or b or c with line numbers. (Use grep/sed)

```
# using grep
```

```
echo -n "Enter file name:"
read filename
```

```
if [ -f "$filename" ]; then
    echo "Lines NOT starting with a, b, or c (with line numbers):"
    grep -n '^[^abc]' "$filename"
else
    echo "File not found!"
fi
```

```
# using sed
```

```
echo "Enter file name:"
```

```
read filename
```

```
if [ -f "$filename" ]; then
    echo "Lines NOT starting with a, b, or c (with line numbers):"
    sed -n '/^[^abc]/=' "$filename" | paste - - | awk '{print $1 ": " $2}'
else
    echo "File not found!"
fi
```

4. Write a script that takes file name from user and substitute all spaces " " with # value. (Use grep/sed)

```
#using sed
```

```
echo "Enter file name:"
read filename

if [ -f "$filename" ]; then
    echo "Output after replacing spaces with #:"
    sed 's/ /#/g' "$filename"
else
    echo "File not found!"
fi
```

5. Write a script that takes file name from user and display all line start with t or T and second character must be either 'h' or 's'. (Use grep/sed)

```
# using grep
```

```
echo -n "Enter file name:"
read filename

if [ -f "$filename" ]; then
    echo "Lines starting with t/T and second char h or s:"
    grep '^[Tt][hs]' "$filename"
else
    echo "File not found!"
fi
```

```
# using sed
```

```
echo -n "Enter file name:"
```

```
read filename
```

```
if [ -f "$filename" ]; then
```

```
    echo "Lines starting with t/T and second char h or s:"
```

```
    sed -n '/^[Tt][hs]/p' "$filename"
```

```
else
```

```
    echo "File not found!"
```

```
fi
```

6. Write a script that takes file name from user and display all line start with space(' ') . (Use grep/sed)

```
# using grep
```

```
echo -n "Enter file name:"
```

```
read filename
```

```
if [ -f "$filename" ]; then
```

```
    echo "Lines starting with a space:"
```

```
    grep ' ' "$filename"
```

```
else
```

```
    echo "File not found!"
```

```
fi
```

```
# using sed
```

```
echo -n "Enter file name:"
```

```
read filename
```

```
if [ -f "$filename" ]; then
```

```
    echo "Lines starting with a space:"
```

```
    sed -n '/^ /p' "$filename"
```

```
else
```

```
    echo "File not found!"
```

```
fi
```

7. Write a script which takes input from a file, with multiple records, as:

Firstname:lastname:address:city:pin:phone

and displays output as:

Record 1

Lastname middlename firstname

Address

City – pin

Phone

Record 2

Lastname middlename firstname

Address

City – pin

Phone

and so on, for all records.

```
echo "Enter file name:"
```

```
read filename
```

```
if [ -f "$filename" ]; then
```

```
    awk -F ':' ' {
```

```
        {
```

```
            print "Record " NR
```

```
            print $3 " " $2 " " $1
```

```
            print $4
```

```
            print $5 " - " $6
```

```
            print $7
```

```
            print ""
```

```
        }' "$filename"
```

```
else
```

```
    echo "File not found!"
```

```
fi
```

8. Write a script that shows usernames and no. of processes running for them.

```
echo "Username   No. of Processes"
```

```
ps -e -o user= | sort | uniq -c | awk '{print $2 "\t\t" $1}'
```

9. The book master file contains the fields book_no, book_name, author, dateofpurchase ,each field is separated by hyphen. write a script for

(A) Add

(b) Modify

(C) Delete

from above file.

```
file="books.txt"
```

```
while true
```

```
do
```

```
    echo "----- MENU -----"
```

```
    echo "1. Add Record"
```

```
    echo "2. Modify Record"
```

```
    echo "3. Delete Record"
```

```
    echo "4. Exit"
```

```
    echo "Enter your choice:"
```

```
    read ch
```

```
    case $ch in
```

```
        1)
```

```
            echo "Enter Book No:"
```

```
            read bno
```

```
            echo "Enter Book Name:"
```

```
            read bname
```

```
            echo "Enter Author:"
```

```
            read author
```

```
            echo "Enter Date of Purchase:"
```

```
            read dop
```

```
            echo "$bno-$bname-$author-$dop" >> $file
```

```
            echo "Record Added!"
```

```
            ;;
```

```
        2)
```

```
            echo "Enter Book No to Modify:"
```

```
            read bno
```

```

if grep -q "^$bno-" $file; then
    echo "Enter New Book Name:"
    read bname
    echo "Enter New Author:"
    read author
    echo "Enter New Date:"
    read dop

    sed -i "s/^$bno-.*/$bno-$bname-$author-$dop/" $file
    echo "Record Modified!"
else
    echo "Record not found!"
fi
;;

3)
echo "Enter Book No to Delete:"
read bno

if grep -q "^$bno-" $file; then
    sed -i "/^$bno-/d" $file
    echo "Record Deleted!"
else
    echo "Record not found!"
fi
;;

4)
exit
;;

*)
echo "Invalid Choice!"
;;

esac
done

```

10. Write a shell script to display list of files which can be either

regular or directory along with number of links in ascending order of links.

```
echo "Files with number of links (ascending order):"
```

```
ls -l | grep -v '^total' | awk '{print $2, $9}' | sort -n
```

11. write a script to count number of vowels in file irrespective of case.

```
echo "Enter file name:"
```

```
read filename
```

```
if [ -f "$filename" ]; then
```

```
    echo "Number of vowels (case-insensitive):"
```

```
    grep -oi '[aeiou]' "$filename" | wc -l
```

```
else
```

```
    echo "File not found!"
```

```
fi
```

12. write a script that accepts a string followed by one or more file names from command line and display no of lines that consists of given string each file

```
# Check minimum arguments
```

```
if [ $# -lt 2 ]; then
```

```
    echo "Usage: $0 <string> <file1> [file2 ...]"
```

```
    exit 1
```

```
fi
```

```
pattern=$1
```

```
shift # remove first argument (string), rest are files
```

```
for file in "$@"
```

```
do
```

```
    if [ -f "$file" ]; then
```

```
        count=$(grep -c "$pattern" "$file")
```

```
        echo "$file : $count"
```

```
    else
```

```
        echo "$file : File not found"
    fi
done
```

13. Create a text file with the headings and data as bill_no, cust_no, cust_name, address, city, pin, current_meter_reading, previous_meter_reading, month. Write a script that takes the input from this file and displays the bill for the query against cust_no/bill_no/cust_name. (input at least 15/20 records in this text file); otherwise message that no record exists

```
file="bill.txt"
```

```
echo "Search by:"
echo "1. Bill No"
echo "2. Customer No"
echo "3. Customer Name"
read choice
```

```
echo "Enter value:"
read value
```

```
case $choice in
1) result=$(grep "^$value:" $file) ;;
2) result=$(grep "^[^:]*:$value:" $file) ;;
3) result=$(grep -i ":$value:" $file) ;;
*) echo "Invalid choice"; exit ;;
esac
```

```
if [ -z "$result" ]; then
    echo "No record exists!"
else
    echo "$result" | while IFS=":" read bno cno cname addr city pin curr prev month
    do
        units=$((curr - prev))
        rate=5 # per unit
        bill=$((units * rate))

        echo "----- ELECTRICITY BILL -----"
        echo "Bill No: $bno"
```

```

        echo "Customer No: $cno"
        echo "Name: $cname"
        echo "Address: $addr"
        echo "City: $city - $pin"
        echo "Month: $month"
        echo "Units Consumed: $units"
        echo "Total Bill: Rs. $bill"
        echo "-----"
    done
fi

```

14. Write a shell script to generate summary from the sales.dat file. Sales made by 3 salesman by selling 3 products are entered in a file. Add atleast 10 records. The format is as shown below:

Salesman:Product1:Product2:Product3

Sample data:

Mr. Abhishek Sharma:21:29:12

Mr. Akash Srivastava:11:15:28

Mr. Abhilash Dwivedi:31:04:13

Calculate the followings :

- 1. Total sales of the company**
- 2. Highest sold product**
- 3. Best salesman (who sold the most)**
- 4. Worst salesman (who sold the least)**

```
file="sales.dat"
```

```

awk -F ':' '
BEGIN {
    total = 0
    p1 = p2 = p3 = 0
    max = -1
    min = 999999
}

{
    salesman = $1
    s1 = $2
    s2 = $3

```

```

s3 = $4

sum = s1 + s2 + s3

total += sum

p1 += s1
p2 += s2
p3 += s3

if (sum > max) {
    max = sum
    best = salesman
}

if (sum < min) {
    min = sum
    worst = salesman
}
}

END {
    print "Total Sales of Company:", total

    if (p1 > p2 && p1 > p3)
        print "Highest Sold Product: Product1"
    else if (p2 > p1 && p2 > p3)
        print "Highest Sold Product: Product2"
    else
        print "Highest Sold Product: Product3"

    print "Best Salesman:", best
    print "Worst Salesman:", worst
}
' "$file"

```

15. Write a shell script to generate summary from a file :

“student.dat” with following format :

**Student_no : student_name : gender : marks1 : marks2 :
marks3**

Each field must be separated by a delimiter ‘-‘

Process the following queries:

- 1. Calculate the total marks of each student**
- 2. Calculate the percentage of marks for each student**
- 3. Count the total number of male and female students**
- 4. Count the total number of students who pass and those who fail.**

```
file='student.dat'
```

```
awk -F ' ' '
```

```
BEGIN {
```

```
    male = female = 0
```

```
    pass = fail = 0
```

```
    print "Student Summary "
```

```
    print "-----"
```

```
}
```

```
{
```

```
    total = $4 + $5 + $6
```

```
    percent = total/3
```

```
    print "Student:",$2
```

```
    print "Total marks:",total
```

```
    printf "Percentage:%.2f\n",percent
```

```
    if($3 == "M")
```

```
        male++
```

```
    else if ($3 == "F")
```

```
        female++
```

```
    if ($4 >= 40 && $5 >= 40 && $6 >= 40)
```

```
        pass++
```

```
    else
```

```
        fail++
```

```
    print "-----"
```

```
}
```

```
END {
```

```
    print "Total Male Students:", male
```

```
    print "Total Female Students:", female
```

```
    print "Total Pass Students:", pass
```

```
    print "Total Fail Students:", fail
```

```
}
' $file
```

16. Create a file employee.dat with fields:

emp_id:emp_name:department:basic_salary:hra:da:pf

(Insert at least 15 records)

Write a shell script to:

Calculate gross salary (basic + hra + da)

Calculate net salary (gross - pf)

Display details of employee based on emp_id / emp_name

Display highest paid employee

```
file="employee.dat"
```

```
echo "Search by:"
```

```
echo "1. Emp ID"
```

```
echo "2. Emp Name"
```

read choice

```
echo "Enter value:"
```

read value

```
awk -F ':' -v ch=$choice -v val="$value" '

```

BEGIN {

max = -1

}

 $\{$

emp_id = \$1

name = \$2

dept = \$3

basic = \$4

hra = \$5

da = \$6

pf = \$7

$$\text{gross} = \text{basic} + \text{hra} + \text{da}$$
$$\text{net} = \text{gross} - \text{pf}$$

Find highest paid employee

```

if (net > max) {
    max = net
    max_emp = name
}

# Search logic
if ((ch == 1 && emp_id == val) || (ch == 2 && name == val)) {
    print "----- Employee Details -----"
    print "ID:", emp_id
    print "Name:", name
    print "Department:", dept
    print "Gross Salary:", gross
    print "Net Salary:", net
    print "-----"
    found = 1
}
}

END {
    if (found != 1)
        print "No record found!"

    print "Highest Paid Employee:", max_emp, "Salary:", max
}
' "$file"

```

17. Electricity Bill Summary

Create a file **electricity.dat** with fields:

consumer_no:name:units_consumed:month

Write a shell script to:

Calculate bill using slab:

0–100 → ₹2/unit

101–200 → ₹3/unit

Above 200 → ₹5/unit

Display total revenue collected

Find consumer with highest bill

Search record by consumer_no/name

```
file="electricity.dat"
```

```
echo "Search by:"
```

```
echo "1. Consumer No"
```

```
echo "2. Name"
```

```
read ch
```

```
echo "Enter value:"
```

```
read val
```

```
awk -F ':' -v choice=$ch -v value="$val" ' '
```

```
BEGIN {
```

```
    total_revenue = 0
```

```
    max_bill = -1
```

```
}
```

```
{
```

```
    cno = $1
```

```
    name = $2
```

```
    units = $3
```

```
    month = $4
```

```
# Slab calculation
```

```
if (units <= 100)
```

```
    bill = units * 2
```

```
else if (units <= 200)
```

```
    bill = (100 * 2) + ((units - 100) * 3)
```

```
else
```

```
    bill = (100 * 2) + (100 * 3) + ((units - 200) * 5)
```

```
total_revenue += bill
```

```
# Highest bill
```

```
if (bill > max_bill) {
```

```
    max_bill = bill
```

```
    max_name = name
```

```
}
```

```
# Search
```

```
if ((choice == 1 && cno == value) || (choice == 2 && name == value)) {
```

```
    print "----- Bill Details -----"
```

```
    print "Consumer No:", cno
```

```
    print "Name:", name
```

```
    print "Units:", units
```

```

        print "Month:", month
        print "Bill Amount: Rs.", bill
        print "-----"
        found = 1
    }
}

END {
    if (found != 1)
        print "No record found!"

    print "Total Revenue: Rs.", total_revenue
    print "Highest Bill Consumer:", max_name, "Amount:", max_bill
}
' "$file"

```

18. Library Management System

Create a file library.dat with fields:

book_id:title:author:student_name:issue_date:return_date

Write a shell script to:

Display all issued books

Find books issued to a particular student_name

Count total issued books

Display overdue books (assume current date input)

```
file="library.dat"
```

```
echo "Enter student name to search:"
```

```
read sname
```

```
echo "Enter current date (DD-MM-YYYY):"
```

```
read today
```

```
echo "----- All Issued Books -----"
```

```
cat "$file"
```

```
echo "----- Books issued to $sname -----"
```

```
grep -i ":$sname:" "$file"
```

```
echo "----- Total Issued Books -----"
```

```
wc -l < "$file"
```

```
echo "----- Overdue Books -----"
```

```
awk -F ':' -v today="$today" '
```

```
function to_days(d) {  
    split(d, a, "-")  
    return a[3]*365 + a[2]*30 + a[1]  
}
```

```
BEGIN {  
    today_days = to_days(today)  
}
```

```
{  
    return_date = $6  
    r_days = to_days(return_date)
```

```
    if (r_days < today_days) {  
        print "Book:", $2, "| Student:", $4, "| Return Date:", $6  
    }  
}  
' "$file"
```

19. Product Inventory Analysis

Create a file inventory.dat with fields:

product_id:product_name:quantity:price

Write a shell script to:

Calculate total inventory value

Find product with highest quantity

Find product with lowest price

Search product by product_id/name

```
file="inventory.dat"
```

```
echo "Search by:"  
echo "1. Product ID"  
echo "2. Product Name"  
read ch
```

```
echo "Enter value:"
```

read val

```
awk -F ':' -v choice=$ch -v value="$val" '
```

```
BEGIN {
```

```
    total_value = 0
```

```
    max_qty = -1
```

```
    min_price = 999999
```

```
}
```

```
{
```

```
    pid = $1
```

```
    pname = $2
```

```
    qty = $3
```

```
    price = $4
```

```
    value_prod = qty * price
```

```
    total_value += value_prod
```

```
    # Highest quantity
```

```
    if (qty > max_qty) {
```

```
        max_qty = qty
```

```
        max_product = pname
```

```
    }
```

```
    # Lowest price
```

```
    if (price < min_price) {
```

```
        min_price = price
```

```
        min_product = pname
```

```
    }
```

```
    # Search
```

```
    if ((choice == 1 && pid == value) || (choice == 2 && pname == value)) {
```

```
        print "----- Product Details -----"
```

```
        print "ID:", pid
```

```
        print "Name:", pname
```

```
        print "Quantity:", qty
```

```
        print "Price:", price
```

```
        print "Total Value:", value_prod
```

```
        print "-----"
```

```
        found = 1
```

```
    }
```

```

}

END {
    if (found != 1)
        print "No record found!"

    print "Total Inventory Value:", total_value
    print "Highest Quantity Product:", max_product, "Qty:", max_qty
    print "Lowest Price Product:", min_product, "Price:", min_price
}
' "$file"

```

20. Cricket Score Analysis

Create a file cricket.dat with fields:

player_name:runs:balls:fours:sixes

Write a shell script to:

Calculate strike rate of each player

Find highest scorer

Count players scoring above 50

Display summary of all players

```
file="cricket.dat"
```

```

awk -F ':' '
BEGIN {
    max_runs = -1
    count50 = 0

    print "----- Player Summary -----"
}

{
    name = $1
    runs = $2
    balls = $3
    fours = $4
    sixes = $5

    # Strike Rate
    strike_rate = (runs / balls) * 100

```

```
printf "Player: %s | Runs: %d | Balls: %d | SR: %.2f\n", name, runs, balls, strike_rate

# Highest scorer
if (runs > max_runs) {
    max_runs = runs
    top_player = name
}

# Count players above 50
if (runs > 50)
    count50++
}

END {
    print "-----"
    print "Highest Scorer:", top_player, "Runs:", max_runs
    print "Players scoring above 50:", count50
}
' "$file"
ss
```